PoUW AppChain Technical Whitepaper v2.0

Specification v2.0 · 9 October 2026

Revision 2.0 integrates four security patches: the final NIST post-quantum suite (Section 2), the sixteen-field anti-grinding seed (Section 5.1), the ShareID-indexed sequential D'Hondt scheduler (Section 6) and Base Fee collection at `JOB_ADMIT_AND_LEASE` (Section 7.7).

Abstract

PoUW AppChain is an application-specific post-quantum layer-1. Verified Fresh Work is the sole source of chain weight, reward rights and execution rights. It is a chain-bound product of two 512×512 integer matrices over $\mathbb{F}_p$, $p = 2^{61} - 1$. Commercial computation carries zero consensus weight.

Correctness is certified by a mixed-field Circle STARK with challenges in $\mathbb{F}_{p^8} = \mathbb{F}_p[X]/(X^8 + X + 9)$ and by six Freivalds rounds, which give a static error of at most $p^{-6} \approx 2^{-366}$. The Fresh Work seed binds sixteen context fields before computation. One evaluation feeds a seal lottery, which carries chain weight, and a share lottery, which carries rights.

Execution slots are allocated by the sequential D'Hondt method over discrete ShareIDs. The allocation is complete, proportional in the limit and invariant under aggregation. Every consensus node reconstructs the rate-1/8 Reed–Solomon data layer.

The hybrid UTXO and shielded-note state machine conserves a fixed supply of 1.5×10^{15} atoms, that is 15,000,000 TKN. The Base Fee moves irretrievably to the locked reserve at admission and is recycled; self-dealing therefore loses at least $(1-\bar{\alpha}\beta)F$ in expectation. Signatures, envelopes and commitments use ML-DSA-87, ML-KEM-1024 and SHA3-512 from Genesis.

Keywords: proof of useful work, Circle STARK, Freivalds' algorithm, FruitChains, D'Hondt method, data availability, ML-DSA, ML-KEM, SHA3-512, UTXO, zero knowledge.

Table of Contents

- Abstract
- 1. Introduction and the PoUW AppChain Model
  - 1.1. Design Thesis · 1.2. Architecture · 1.3. Normative Parameter Core · 1.4. Encoding Conventions · 1.5. Adversary Model
- 2. Post-Quantum Cryptography Foundation
  - 2.1. Genesis Suite · 2.2. Signatures and Authorization · 2.3. KEM–DEM Envelopes · 2.4. Digest Length against Grover and BHT · 2.5. Post-Quantum Soundness of the Proof Layer
- 3. Module 1 — Compute Primitive and Circle STARK
  - 3.1. Canonical Tile · 3.2. Freivalds Verification · 3.3. Mixed-Field Circle STARK · 3.4. Error Budget and Tile Decomposition · 3.5. Prover Memory
- 4. Module 2 — Prover Data Availability Layer (P-DAL)
  - 4.1. Encoding and Commitment · 4.2. Data Availability Sampling · 4.3. Reconstruction Rule · 4.4. Fast-Path Certificate F1
- 5. Module 3 — Dual-Lottery Consensus and Anti-Grinding Fresh Work
  - 5.1. Fresh Work Context and Seed · 5.2. Instance Generation and Identifiers · 5.3. Dual-Lottery · 5.4. Anti-Grinding Theorem · 5.5. Fork Choice · 5.6. Delay and Reorganization Bound · 5.7. Rewards and Deviation Bound
- 6. Module 4 — Deterministic Scheduler and D'Hondt Allocation

  - 6.1. ShareID-Indexed Weights · 6.2. Floor-Quota Defect · 6.3. Sequential D'Hondt Recursion · 6.4. Completeness and Proportionality · 6.5. Aggregation Invariance · 6.6. Leases and Phase Machine · 6.7. Ordering and Throughput
- 7. Module 5 — Hybrid State, zk-TWL and Tokenomics
  - 7.1. State and Operation Set · 7.2. TWL Authorization · 7.3. zk-TWL Notes · 7.4. Private Funding and Disclosure Set · 7.5. Atomic Transitions and Settlement · 7.6. Supply Invariant · 7.7. Fee Schedule and Anti-Spam Base Fee · 7.8. Recycling and Reserve Dynamics · 7.9. Self-Dealing Theorem · 7.10. Resource Limits and L2 Slots
- 8. Conclusion
  - 8.1. Guarantees · 8.2. Conformance Invariants · 8.3. Concluding Statement
- References

1. Introduction and the PoUW AppChain Model

1.1. Design Thesis

Chain weight, economic rights and commercial payload are produced by disjoint maps. Fresh Work [22]–[26] enters only the first two, and the payload enters neither (1.1). A buyer of computation therefore cannot purchase weight, and a miner cannot convert payments into fork-choice influence.

$$\mathcal{W} = \mathcal{W}(\mathrm{seals}(\mathcal{C})), \qquad \mathsf{Rights} = \mathsf{Rights}(\mathcal{S}_{\mathrm{mature}}), \qquad \mathrm{seals}(\mathcal{C}) = \mathrm{seals}(\mathcal{C}') \;\Rightarrow\; \mathcal{W}(\mathcal{C}) = \mathcal{W}(\mathcal{C}') \tag{1.1}$$

Legend:

- $\mathcal{C}, \mathcal{C}'$ — chains that may differ in commercial results, fees and bonds; $\mathrm{seals}(\mathcal{C})$ — the multiset of Fresh Work seals of $\mathcal{C}$;
- $\mathcal{W}$ — the cumulative chain weight (Section 5.5);
- Rights — reward and execution entitlements; $\mathcal{S}_{\mathrm{mature}}$ — the mature Fresh Work shares.

1.2. Architecture

The protocol is a post-quantum foundation PQC composed with five modules $\mathsf{M}_1, \ldots, \mathsf{M}_5$. Three flows connect them (1.2): the Fresh Work flow, the job flow and the fee flow.

$$\begin{aligned} & x \xrightarrow{\mathsf{M}_1} (\mathit{outputRoot},\ \pi_{\mathrm{FW}}) \xrightarrow{\mathsf{M}_3} (\vartheta_B,\ \vartheta_S) \longmapsto (\mathcal{W},\ \mathsf{Rights}) \\ & \mathsf{Escrow} \xrightarrow{\mathsf{M}_4} \mathsf{Lease} \xrightarrow{\mathsf{M}_1,\ \mathsf{M}_2} (Z,\ \pi,\ \text{available data}) \xrightarrow{\mathsf{M}_5} \Sigma' \\ & F^{\mathrm{base}} \longrightarrow R_l \xrightarrow{\text{maturation}} R_a \longrightarrow \text{Fresh Work rewards} \end{aligned} \tag{1.2}$$

Legend:

- M_1 — compute primitive and Circle STARK; M_2 — prover data availability layer; M_3 — Dual-Lottery consensus; M_4 — deterministic scheduler; M_5 — hybrid state, zk-TWL and tokenomics;
- x — the Fresh Work context; $\mathit{outputRoot}, \pi_{\mathrm{FW}}$ — the committed result and its proof; ϑ_B, ϑ_S — the lottery values;
- Z, π — a commercial result and its proof; Σ' — the settled state;
- F^{base} — the Base Fee; R_l, R_a — the locked and the available reserve.

1.3. Normative Parameter Core

The values in Table 1 are normative for all modules. Every other quantity is a deployment parameter of the versioned release manifest.

Table 1. Normative parameter core.

Parameter	Value	Defined in
Base field	$p = 2^{61} - 1 = 2\,305\,843\,009\,213\,693\,951$	3.1
Challenge field	$\mathbb{F}_{p^8} = \mathbb{F}_p[X]/(X^8 + X + 9)$	3.3
Native tile	$n = 512$; INT8 inputs in $[-127, 127]$; INT32 result	3.1
Freivalds rounds	$r_F = 6$; static error $\leq p^{-6} \approx 2^{-366}$	3.2
Prover memory	peak ≤ 19.5 GiB	3.5
Signatures	ML-DSA-87 (FIPS 204); treasury: ML-DSA-87 $\wedge$ SLH-DSA-SHAKE-256s (FIPS 205)	2.2
Envelopes	ML-KEM-1024 (FIPS 203), HKDF-SHA-512, AES-256-GCM	2.3
Hash, XOF, PRF	SHA3-512, SHAKE256, KMAC256	2.4
Fresh Work seed	$seed = \mathsf{SHA3\text{-}512}(\texttt{FW/INPUT/v2}, x)$, with x of 16 fields	5.1
P-DAL	Reed–Solomon, rate 1/8; full reconstruction by consensus nodes	4.1
Nominal rates	$T_B = 120$ s; $\lambda_S = 1$ s^{-1}	5.3
Scheduler	sequential D'Hondt, quotient $w_s/(a_s + 1)$ over ShareIDs	6.3
Supply	$M = 1.5 \times 10^{15}$ atoms = 15,000,000 TKN	7.6
Genesis	10% team, 90% Network Emission Reserve	7.6
Base Fee	irretrievable transfer to R_l at `JOB_ADMIT_AND_LEASE`; recycled, never burned	7.7
Commercial weight	0	1.1

1.4. Encoding Conventions

Every hashed object is domain-separated and injectively encoded, and every consensus quantity is an integer, (1.3)–(1.4). Fractional expressions such as $2^{512}/\Theta_B$ are analytic notation for integer computations.

$$H_{512} := \mathsf{SHA3\text{-}512}, \qquad digest = H_{512}(\mathsf{Enc}(\texttt{LABEL/v2},\ f_1, \dots, f_k)), \qquad \mathsf{Enc}\ \text{injective} \qquad (1.3)$$

$$\frac{a}{b} < \frac{c}{d} \iff a\,d < c\,b \quad (a, b, c, d \in \mathbb{Z},\ b, d > 0) \qquad (1.4)$$

Legend:

- H_{512} — SHA3-512; *digest* — any 512-bit commitment, identifier or lottery value, read as a big-endian integer;
- Enc — the typed, length-prefixed encoding with fixed field order that rejects non-canonical forms;
- `LABEL/v2` — a versioned domain label, distinct for every use; $f_1, \dots, f_k$ — the encoded fields;
- (1.4) — the exact comparison rule for ratios; no consensus rule uses floating point.

1.5. Adversary Model

Every theorem in Sections 2 to 7 holds in the model of Table 2.

Table 2. Model assumptions.

Assumption	Formal statement	Used in
A1 — Fresh Work	$\mathrm{cost}_{\mathcal{A}}(\text{attempt on fresh } x) \gtrsim \mathrm{cost}(\mathsf{Eval}_{\mathrm{FW}})$; residual speed-ups, including quantum ones, enter α	5.4
A2 — Bounded delay	$t_{\text{discovery}\to\text{validation}} \leq \Delta_{\text{eff}}$, as a bound and not a mean	5.6
A3 — Honest advantage	$u = (1-\alpha)\exp(-\Delta_{\text{eff}}/T_B) > 1/2$ at constant rates; α is the adversarial share of effective seal capacity	5.6, 6.7
A4 — Proof profile	$\varepsilon_{\mathrm{STARK}} + \varepsilon_{\mathrm{commit}} \leq 2^{-128}$ over the complete versioned profile in the QROM; quantum proof of knowledge and quantum zero knowledge for the note profile	2.5, 3.4, 7.3
A5 — Fairness event	$\mathcal{F}_\delta$: every honest group of share φ receives at least $(1-\delta_f)\varphi E_e$ [1], [2]	5.7, 7.9

Assumption	Formal statement	Used in
A6 — Static sampling	The withheld set is fixed and samples are drawn without replacement; no selective responses, query observation or isolation	4.2
A7 — Quantum interface	The adversary computes quantumly; honest signing and decapsulation interfaces are classical	2.3, 2.4

2. Post-Quantum Cryptography Foundation

2.1. Genesis Suite

The Genesis suite consists of final NIST standards [38]–[46]. No component's security reduces to factoring or discrete logarithms (2.1)–(2.2). Suites change only through a versioned consensus rule, which excludes downgrade. Parameter sets and object sizes are given in Table 3.

Table 3. Genesis parameter sets and sizes in bytes.

Function	Scheme, NIST category	Sizes
Operation signatures	ML-DSA-87, category 5	public key 2592; signature 4627
Treasury co-signature	SLH-DSA-SHAKE-256s, category 5	public key 64; signature 29,792
Envelope key establishment	ML-KEM-1024, category 5	encapsulation key 1568; ciphertext 1568; secret 32
Content encryption	AES-256-GCM	key 32; nonce 12; tag 16
Commitments, identifiers, lotteries	SHA3-512	digest 64
Expansion and keyed derivation	SHAKE256, KMAC256	PRF output 64

$$\begin{aligned}\mathsf{Suite}_{\mathrm{Gen}} = (&\mathsf{ML\text{-}DSA\text{-}87}_{[\mathrm{FIPS\ 204}]},\ \mathsf{SLH\text{-}DSA\text{-}SHAKE\text{-}256s}_{[\mathrm{FIPS\ 205}]},\\ &\mathsf{ML\text{-}KEM\text{-}1024}_{[\mathrm{FIPS\ 203}]},\ \mathsf{AES\text{-}256\text{-}GCM}_{[\mathrm{SP\ 800\text{-}38D}]},\ \mathsf{HKDF\text{-}SHA\text{-}512}_{[\mathrm{RFC\ 5869}]},\\ &\mathsf{SHA3\text{-}512},\ \mathsf{SHAKE256}_{[\mathrm{FIPS\ 202}]},\ \mathsf{KMAC256}_{[\mathrm{SP\ 800\text{-}185}]})\end{aligned} \tag{2.1}$$

$$\begin{aligned}&\mathsf{Suite}_{\mathrm{Gen}} \cap \{\text{ECDSA, Schnorr, BLS, Pedersen, KZG, pairings, Groth16, DH}\} = \emptyset\\ &obj.cryptoSuiteID \in \mathcal{K}_{\mathrm{active}}(h)\end{aligned} \tag{2.2}$$

Legend:

- $\mathsf{Suite}_{\mathrm{Gen}}$ — the closed Genesis suite; subscripts name the defining standard;
- the excluded set — every primitive whose security reduces to factoring or discrete logarithms;
- *obj* — any signed, encrypted or committed protocol object; $\mathcal{K}_{\mathrm{active}}(h)$ — the set of suites active at height h, changed only by a versioned consensus rule.

2.2. Signatures and Authorization

Every public spend carries a pure-mode ML-DSA-87 signature over the complete effect digest, and the presented key must match the locking policy read from state (2.3)–(2.4); shielded spends are authorized by the proof of Section 7.3. Treasury and upgrade outputs require the conjunction with SLH-DSA-SHAKE-256s under independent keys (2.5); a disjunction is not admitted.

$$AuthMsg = \mathsf{Enc}(\texttt{PoUW/TX/v2},\ TxID), \qquad \sigma \leftarrow \mathsf{ML\text{-}DSA\text{-}87.Sign}(sk,\ AuthMsg,\ ctx) \tag{2.3}$$

$$\mathsf{Accept}(tx) \iff \mathsf{ML\text{-}DSA\text{-}87.Verify}(pk,\ AuthMsg,\ \sigma,\ ctx) = 1 \ \wedge\ \mathsf{Match}(pk,\ lockingPolicy_{\mathrm{prev}}) = 1 \tag{2.4}$$

$$\begin{aligned}\mathsf{Accept}_{\text{treasury}}(tx) \iff\ & \mathsf{ML\text{-}DSA\text{-}87.Verify}(pk_1,\ AuthMsg,\ \sigma_1,\ ctx) = 1 \\ & \wedge\ \mathsf{SLH\text{-}DSA\text{-}SHAKE\text{-}256s.Verify}(pk_2,\ AuthMsg,\ \sigma_2,\ ctx) = 1 \\ & \wedge\ \mathsf{Match}((pk_1,\ pk_2),\ lockingPolicy_{\text{prev}}) = 1\end{aligned} \tag{2.5}$$

Legend:

- tx — the transaction; $TxID$ — its effect identifier (7.5); $AuthMsg$ — the signed message;
- $ctx = \texttt{PoUW/TX/v2}$ — the context string of FIPS 204 and FIPS 205;
- sk, pk — an ML-DSA-87 key pair; $\sigma, \sigma_1, \sigma_2$ — signatures;
- pk_1, pk_2 — independent ML-DSA-87 and SLH-DSA-SHAKE-256s keys;
- $lockingPolicy_{\text{prev}}$ — the policy of the consumed output, read from state; Match — key-to-policy conformance.

2.3. KEM–DEM Envelopes

Note openings and confidential payloads are sealed under ML-KEM-1024 with a fresh encapsulation per envelope, HKDF-SHA-512 and AES-256-GCM (2.6) [41], [44], [45]. The recipient key ek_R is authenticated by an ML-DSA-87 signature. Freshness excludes key–nonce reuse (2.7).

$$\begin{aligned}(kemCT,\ ss) &\leftarrow \mathsf{ML\text{-}KEM\text{-}1024.Encaps}(ek_R) \\ ectx &= \mathsf{Enc}(chainID,\ suiteID,\ noteCommitment,\ outputIndex,\ recipientKeyEpoch,\ purpose) \\ prk &= \mathsf{HKDF\text{-}Extract}(H_{512}(\mathsf{Enc}(\texttt{ENVELOPE/SALT/v2},\ ectx,\ kemCT)),\ ss) \\ k &= \mathsf{HKDF\text{-}Expand}(prk,\ \mathsf{Enc}(\texttt{ENVELOPE/KEY/v2},\ ectx),\ 32), \qquad nonce \xleftarrow{\text{R}} \{0,1\}^{96} \\ (ct,\ tag) &\leftarrow \mathsf{AES\text{-}256\text{-}GCM.Seal}(k,\ nonce,\ m,\ \mathsf{Enc}(ectx,\ kemCT,\ nonce))\end{aligned} \tag{2.6}$$

$$i \neq j\ \Rightarrow\ (k_i,\ nonce_i) \neq (k_j,\ nonce_j) \tag{2.7}$$

Legend:

- ek_R — the authenticated encapsulation key of the recipient; $kemCT, ss$ — the KEM ciphertext and the 32-byte shared secret;
- $ectx$ — the envelope context, which binds purpose and note commitment but not $TxID$ (no circular definition);
- prk, k — the HKDF pseudorandom key and the 32-byte AEAD key; HKDF is instantiated with SHA-512; m — the plaintext;
- ct, tag — the ciphertext and the 128-bit tag; $nonce$ — a fresh 96-bit value;
- i, j — envelope indices.

2.4. Digest Length against Grover and BHT

Grover search [47] inverts an n-bit hash with about $2^{n/2}$ queries, and the Brassard–Høyer–Tapp algorithm [48] finds a collision with about $2^{n/3}$ queries (2.8). Only digests of at least 384 bits keep the quantum collision cost at the 128-bit target, so SHA3-512 is used for every Merkle root, commitment, transaction identifier, work identifier and lottery value (2.9). Inside the AIR, a digest is carried as sixteen range-checked 32-bit limbs and is never reduced modulo p (2.10).

$$Q_{\text{pre}}(n) \sim 2^{n/2}, \qquad Q_{\text{col}}(n) \sim 2^{n/3} \tag{2.8}$$

$$Q_{\text{col}}(256) \approx 2^{85.3} < 2^{128}, \qquad Q_{\text{col}}(512) \approx 2^{170.7}, \qquad Q_{\text{pre}}(512) \approx 2^{256} \tag{2.9}$$

$$digest = \sum_{j=0}^{15} \ell_j \, 2^{32j}, \qquad 0 \le \ell_j < 2^{32} \qquad (2.10)$$

Legend:

- n — the digest length in bits; $Q_{\mathrm{pre}}, Q_{\mathrm{col}}$ — the idealized quantum query costs of preimage and collision search;
- ℓ_j — the 32-bit limbs of a 512-bit digest in the AIR; a single element of $\mathbb{F}_p$ would bind only about 61 bits.

2.5. Post-Quantum Soundness of the Proof Layer

Circle STARKs [12] use the circle group over $\mathbb{F}_p$, whose order for $p = 2^{61} - 1$ is 2^{61} (2.11). Soundness rests on hash binding and on algebraic tests; Shor's algorithm [49] applies to neither. The error decomposes additively over the binding, the Fiat–Shamir and FRI analysis in the QROM [50], and the algebraic tests (2.12) (A4).

$$C(\mathbb{F}_p) = \{(u, v) \in \mathbb{F}_p^2 : u^2 + v^2 = 1\}, \qquad |C(\mathbb{F}_p)| = p + 1 = 2^{61} \qquad (2.11)$$

$$\varepsilon_{\mathrm{STARK}} \le \varepsilon_{\mathrm{bind}} + \varepsilon_{\mathrm{QROM}} + \varepsilon_{\mathrm{alg}} \qquad (2.12)$$

Legend:

- $C(\mathbb{F}_p)$ — the circle group; $|C(\mathbb{F}_p)| = p + 1$ because $p \equiv 3 \pmod 4$;
- $\varepsilon_{\mathrm{bind}}$ — the probability of breaking the quantum binding of commitments and Merkle trees;
- $\varepsilon_{\mathrm{QROM}}$ — the soundness error of Fiat–Shamir and FRI in the QROM, including reduction losses and admitted attempts;
- $\varepsilon_{\mathrm{alg}}$ — the error of the algebraic tests over $\mathbb{F}_{p^8}$ (3.12).

3. Module 1 — Compute Primitive and Circle STARK

3.1. Canonical Tile

The native work unit is an exact integer product, (3.1). Its range excludes modular aliasing in $\mathbb{F}_p$, (3.2)–(3.3). Under the classical algorithm it costs n^3 multiply–accumulate operations, (3.4). The value -128 is excluded, because $512 \cdot 128^2 = 2^{23}$ would violate the strict bound.

$$X, Y \in \{-127, \dots, 127\}^{512 \times 512}, \qquad Z = XY \in \mathbb{Z}^{512 \times 512} \qquad (3.1)$$

$$|Z_{ij}| \le 512 \cdot 127^2 = 8\,258\,048 < 2^{23} \qquad (3.2)$$

$$|Z_{ij} - Z'_{ij}| < 2^{24} < p \;\Rightarrow\; (Z_{ij} \equiv Z'_{ij} \pmod p \iff Z_{ij} = Z'_{ij}) \qquad (3.3)$$

$$C_{\mathrm{mul}} = n^3 = 512^3 = 2^{27} \qquad (3.4)$$

Legend:

- X, Y — the INT8 input matrices; Z — the canonical INT32 product; Z_{ij} — an entry;
- Z' — any admissible result, that is, any matrix satisfying (3.2);
- $p = 2^{61} - 1$ — the base modulus; $n = 512$ — the tile dimension;
- C_{mul} — the classical multiply–accumulate count of one tile.

3.2. Freivalds Verification

Six independent rounds over $\mathbb{F}_p$ bound the acceptance of a fixed false product by p^{-6} (3.5)–(3.8) [8], [9]. The vectors are derived after X, Y and Z are bound, from a versioned Fiat–Shamir transcript [10] with one domain per round and rejection sampling. The choice $r_F = 6$ raises the quantum search cost to 2^{183} (3.9).

$$v^{(\ell)} \xleftarrow{\mathrm{R}} \mathbb{F}_p^{\,n}, \qquad X(Yv^{(\ell)}) = Zv^{(\ell)}, \qquad \ell = 1, \dots, r_F = 6 \tag{3.5}$$

$$D = XY - Z \neq 0 \;\Rightarrow\; \Pr[Dv = 0] = p^{-\operatorname{rank}(D)} \leq p^{-1} \tag{3.6}$$

$$\Pr[\text{accept} \mid Z \neq XY] \leq p^{-6} \approx 2^{-366}, \qquad \Pr[\text{accept within } Q \text{ attempts}] \leq Q\,p^{-6} \tag{3.7}$$

$$C_{\mathrm{Frei}} = 3\,r_F\,n^2 = 4\,718\,592 = \tfrac{9}{256}\,C_{\mathrm{mul}} \tag{3.8}$$

$$Q_{\mathrm{search}}^{\mathrm{Frei}} \sim p^{r_F/2} = p^3 \approx 2^{183} \tag{3.9}$$

Legend:

- $v^{(\ell)}$ — the challenge vector of round ℓ; $\xleftarrow{\mathrm{R}}$ — uniform sampling after binding;
- D — the error matrix; $\operatorname{rank}(D)$ — its rank over $\mathbb{F}_p$;
- Q — the number of false transcripts tested classically;
- C_{Frei} — the field multiplications of the six rounds;
- $Q_{\mathrm{search}}^{\mathrm{Frei}}$ — the quantum search cost of an accepting false transcript.

3.3. Mixed-Field Circle STARK

The trace lives in $\mathbb{F}_p$, and every verifier and FRI challenge lives in $\mathbb{F}_{p^8}$ (3.10) [11], [12], [13]. The defining polynomial is irreducible: the only prime divisor of 8 is 2, so Rabin's test [51] reduces irreducibility to (3.11), and both conditions hold in exact arithmetic. A degree-8 extension lifts the per-check quantum search cost above the 128-bit target, which degree 4 does not reach (3.12)–(3.13). The FRI code rate, folding schedule, queries, openings and masking are fixed by the versioned profile so that A4 holds [14], [15], [16], [52].

$$\mathbb{F}_{p^8} = \mathbb{F}_p[X]/(f), \qquad f(X) = X^8 + X + 9, \qquad \text{basis } 1, X, \dots, X^7 \tag{3.10}$$

$$X^{p^8} \equiv X \pmod{f}, \qquad \gcd(f,\; X^{p^4} - X) = 1 \tag{3.11}$$

$$\varepsilon_{\mathrm{alg}} \lesssim \frac{d}{p^m}, \qquad Q_{\mathrm{search}} \sim \left(\frac{p^m}{d}\right)^{1/2} \tag{3.12}$$

$$\log_2 Q_{\mathrm{search}} \approx 244 - \tfrac{1}{2}\log_2 d \quad (m = 8), \qquad \log_2 Q_{\mathrm{search}} \approx 122 - \tfrac{1}{2}\log_2 d \quad (m = 4) \tag{3.13}$$

Legend:

- X — here the indeterminate of $\mathbb{F}_p[X]$; f — the defining polynomial;
- (3.11) — Rabin's conditions: $f \mid X^{p^8} - X$, and f has no irreducible factor of degree 1, 2 or 4;
- d — the degree of one algebraic check; m — the extension degree;
- $\varepsilon_{\mathrm{alg}}$ — the error of that check; Q_{search} — the quantum search cost of a bad challenge.

3.4. Error Budget and Tile Decomposition

The probability of accepting an incorrect result is bounded by the sum of the component errors, never by their product (3.14). Data availability has the separate budget $\varepsilon_{\mathrm{DA}}$. Products larger than one tile decompose canonically into tile products and certified reductions (3.15)–(3.16). A 1024×1024 product therefore takes eight tile products and four reductions.

$$\varepsilon_{\mathrm{compute}} \le \varepsilon_{\mathrm{STARK}} + \varepsilon_{\mathrm{Freivalds}} + \varepsilon_{\mathrm{commit}} \qquad (3.14)$$

$$Z_{ij}^{(t)} = X_{it}\, Y_{tj}, \qquad C_{ij} = \sum_{t} Z_{ij}^{(t)} \qquad (3.15)$$

$$N_{\mathrm{mul}} = \left\lceil \frac{m}{512} \right\rceil \left\lceil \frac{k}{512} \right\rceil \left\lceil \frac{n_o}{512} \right\rceil \qquad (3.16)$$

Legend:

- $\varepsilon_{\mathrm{compute}}$ — the acceptance probability of an incorrect result;
- $\varepsilon_{\mathrm{STARK}}$ (2.12), $\varepsilon_{\mathrm{Freivalds}}$ (3.7), $\varepsilon_{\mathrm{commit}}$ — the component errors;
- X_{it}, Y_{tj} — 512×512 blocks; $Z_{ij}^{(t)}$ — a partial product; C_{ij} — an output block;
- $m \times k$, $k \times n_o$ — the operand dimensions; N_{mul} — the number of tile products.

3.5. Prover Memory

The conforming native-tile prover peaks at 19.5 GiB, which fits within consumer GPU memory (3.17)–(3.19). The trace width covers every auxiliary, range and lookup column. Extension-field values are accounted in M_{rest}. A trace of 2^{21} rows would double M_{LDE} to 32 GiB, so the primitive is the 512×512 tile.

$$M_{\mathrm{tr}} = N_{\mathrm{tr}}\, W_{\mathrm{tr}} \cdot 8 \text{ B} = 2^{20} \cdot 2^{7} \cdot 2^{3} \text{ B} = 1 \text{ GiB} \qquad (3.17)$$

$$M_{\mathrm{LDE}} = 2\, B_{\mathrm{LDE}}\, N_{\mathrm{tr}}\, W_{\mathrm{tr}} \cdot 8 \text{ B} = 2 \cdot 2^{33} \text{ B} = 16 \text{ GiB} \qquad (3.18)$$

$$M_{\mathrm{peak}} \le M_{\mathrm{tr}} + M_{\mathrm{LDE}} + M_{\mathrm{rest}} \le (1 + 16 + 2.5) \text{ GiB} = 19.5 \text{ GiB} \qquad (3.19)$$

Legend:

- $N_{\mathrm{tr}} = 2^{20}$ — trace rows; $W_{\mathrm{tr}} = 128$ — trace columns; 8 bytes per base element;
- $B_{\mathrm{LDE}} = 8$ — the STARK blow-up factor, distinct from the P-DAL rate;
- $M_{\mathrm{tr}}, M_{\mathrm{LDE}}, M_{\mathrm{rest}}, M_{\mathrm{peak}}$ — the memory of the trace, of the two LDE buffers, of the remaining buffers (at most 2.5 GiB) and the peak.

4. Module 2 — Prover Data Availability Layer (P-DAL)

4.1. Encoding and Commitment

The payload is encoded with a rate-1/8 Reed–Solomon code [17] and committed by a SHA3-512 Merkle tree [18] with distinct leaf and node labels (4.1)–(4.2). A Merkle path proves membership of a fragment, and a proximity proof proves the encoding [13], [19].

$$\mathrm{RS}[n_{\mathrm{RS}},\ k_{\mathrm{RS}}], \qquad \frac{k_{\mathrm{RS}}}{n_{\mathrm{RS}}} = \frac{1}{8}, \qquad k_{\mathrm{RS}} = d + 1 \qquad (4.1)$$

$$root = \mathsf{MerkleRoot}_{H_{512}}(c_1, \ldots, c_{n_{\mathrm{RS}}}), \qquad |A| \ge k_{\mathrm{RS}} \ \Rightarrow\ \mathsf{Decode}(\{c_i\}_{i \in A}) = data \qquad (4.2)$$

Legend:

- $n_{\mathrm{RS}}, k_{\mathrm{RS}}$ — the code length and dimension; d — the maximal polynomial degree;
- c_i — the codeword symbols; *root* — the commitment bound in the manifest;
- A — a set of distinct authenticated symbol indices; Decode — Reed–Solomon reconstruction.

4.2. Data Availability Sampling

Light clients accept availability on the fast path by sampling authenticated fragments after the commitment is fixed [20], [21]. Under A6 the miss probability decays as 2^{-3s} (4.3)–(4.4). For example, $s = 43$ samples give $P_{\mathrm{miss}} < 2^{-129}$.

$$P_{\mathrm{miss}} = \binom{n_{\mathrm{RS}} - w}{s} \Big/ \binom{n_{\mathrm{RS}}}{s} \leq \left(1 - \frac{w}{n_{\mathrm{RS}}}\right)^{s} \tag{4.3}$$

$$n_{\mathrm{RS}} - w < k_{\mathrm{RS}} = \frac{n_{\mathrm{RS}}}{8} \;\Rightarrow\; P_{\mathrm{miss}} < 8^{-s} = 2^{-3s} \tag{4.4}$$

Legend:

- P_{miss} — the probability that every sample hits an available symbol although w symbols are withheld;
- w — the number of withheld symbols; s — the number of samples drawn without replacement;
- $n_{\mathrm{RS}} - w < k_{\mathrm{RS}}$ — the condition for unrecoverable data.

4.3. Reconstruction Rule

Consensus nodes admit a block to the fork choice only after physical reconstruction and full verification (4.5). Sampling success replaces no conjunct.

$$\mathsf{Admit}(\mathsf{blk}) \iff |A| \geq k_{\mathrm{RS}} \;\wedge\; \mathsf{Commit}(\mathsf{Decode}(A)) = \mathit{root} \;\wedge\; \mathsf{VerifyProofs}(\mathsf{blk}) \;\wedge\; \mathsf{VerifyTransitions}(\mathsf{blk}) \tag{4.5}$$

Legend:

- blk — a candidate block; A — the retrieved authenticated fragments;
- Commit — recomputation of the root from the decoded data and their encoding;
- VerifyProofs, VerifyTransitions — verification of every proof and every data-dependent state transition.

4.4. Fast-Path Certificate F1

A result certificate separates a branch-invariant correctness relation from a branch-dependent payment relation (4.6)–(4.7). Delivery latency is bounded below by the remaining proving, transport and verification (4.8). Balance finality follows the confirmation depth of Section 5.6.

$$\mathrm{ComputeValid}_{\mathcal{C}}(\kappa) = \mathrm{ComputeValid}_{\mathcal{C}'}(\kappa) \quad \forall\, \mathcal{C}, \mathcal{C}', \qquad \kappa = (\mathit{program},\ \mathit{inputRoots},\ \mathit{outputRoot},\ \pi) \tag{4.6}$$

$$\mathsf{Pay} > 0 \;\Rightarrow\; \mathrm{PaymentValid}(\Sigma_{\mathcal{C}^*},\ \mathit{jobID},\ \mathit{tileID},\ \mathit{leaseVersion},\ \mathit{executor},\ \mathit{receiptID}) = 1 \tag{4.7}$$

$$t_{\mathrm{ready}} \gtrsim t_{\mathrm{proof,rem}} + t_{\mathrm{transport}} + t_{\mathrm{verify}}, \qquad t_{\mathrm{transport}} \geq \frac{|\text{remaining data}|}{\text{bandwidth}} \tag{4.8}$$

Legend:

- $\mathcal{C}, \mathcal{C}'$ — any two branches; $\mathcal{C}^*$ — the canonical branch (5.10); $\Sigma_{\mathcal{C}^*}$ — its state; Pay — a payout;
- κ — the certificate: the program, the committed inputs and output, and the proof π;
- *jobID*, *tileID*, *leaseVersion*, *executor*, *receiptID* — the identifiers of the economic claim;
- t_{ready} — the time to a verified result at the client; $t_{\mathrm{proof,rem}}, t_{\mathrm{transport}}, t_{\mathrm{verify}}$ — its components.

5. Module 3 — Dual-Lottery Consensus and Anti-Grinding Fresh Work

5.1. Fresh Work Context and Seed

The Internal Oracle is a public, versioned generator anchored in chain state; no external oracle exists. Before any matrix computation, every attempt binds sixteen fields into x, among them *attemptNonce* and *bodyCoreRoot* (5.1). The seed is $\mathsf{SHA3\text{-}512}(\texttt{FW/INPUT/v2}, x)$ (5.2).

$$\begin{aligned} x = \mathsf{Enc}(&\mathit{chainID},\ \mathit{genesisID},\ \mathit{protocolVersion},\ \mathit{cryptoSuiteID},\\ &\mathit{programID},\ \mathit{AIR_ID},\ \mathit{generatorVersion},\\ &\mathit{parentCanonicalID},\ \mathit{stableAnchorID},\ \mathit{height},\\ &\mathit{minerPolicyDigest},\ \mathit{payoutPolicyDigest},\ \mathit{attemptNonce},\\ &\mathit{bodyCoreRoot},\ \mathit{parentDerivedDifficultyContext},\ \mathit{timeBucket}) \end{aligned} \tag{5.1}$$

$$\mathit{seed} = H_{512}(\mathsf{Enc}(\texttt{FW/INPUT/v2},\ x)) \tag{5.2}$$

Legend:

- *chainID*, *genesisID*, *protocolVersion*, *cryptoSuiteID* — the chain, Genesis, protocol and suite domain;
- *programID*, *AIR_ID*, *generatorVersion* — the program, the AIR and the generator version;
- *parentCanonicalID*, *stableAnchorID*, *height* — the parent, the stable anchor and the height;
- *minerPolicyDigest*, *payoutPolicyDigest* — the miner and payout policies, which fix the beneficiary before work;
- *attemptNonce* — the nonce; *bodyCoreRoot* — the body root fixed before the attempt, excluding the current proof, result and receipt;
- *parentDerivedDifficultyContext*, *timeBucket* — the difficulty context and the time bucket.

5.2. Instance Generation and Identifiers

SHAKE256 expands the seed into uniform INT8 matrices by byte rejection (5.3). The work identifier binds the context and the committed output, and it serves as the ShareID (5.4).

$$\begin{aligned} X = \mathsf{Map}(\mathsf{SHAKE256}(\mathsf{Enc}(\texttt{FW/A},\ \mathit{seed}))), \qquad Y = \mathsf{Map}(\mathsf{SHAKE256}(\mathsf{Enc}(\texttt{FW/B},\ \mathit{seed})))\\ \mathsf{Map}(b) = b - 127 \quad \text{for } b \in \{0, \dots, 254\}, \qquad b = 255 \text{ rejected} \end{aligned} \tag{5.3}$$

$$\begin{aligned} \mathit{outputRoot} = H_{512}(\mathsf{Enc}(\texttt{FW/OUTPUT/v2},\ XY)), \qquad \mathit{WorkID} = H_{512}(\mathsf{Enc}(\texttt{FW/ID/v2},\ x,\ \mathit{outputRoot}))\\ \mathit{BlockID} = H_{512}(\mathsf{Enc}(\texttt{BLOCK/v2},\ x,\ \mathit{outputRoot})), \qquad \mathit{ShareID} = \mathit{WorkID} \end{aligned} \tag{5.4}$$

Legend:

- Map — the byte map; $b = 255$ is rejected, so entries are exactly uniform on $[-127, 127]$;
- $\texttt{FW/A}, \texttt{FW/B}, \texttt{FW/OUTPUT/v2}, \texttt{FW/ID/v2}, \texttt{BLOCK/v2}$ — domain labels;
- *outputRoot* — the commitment to the canonical product XY;
- *WorkID* = *ShareID* — the work identifier; *BlockID* — the identifier of the block the attempt would seal.

5.3. Dual-Lottery

One verified evaluation enters two domain-separated lotteries in the style of FruitChains [1] (5.5)–(5.6). The rare seal decides the fork choice and absorbs Δ_{eff}. The frequent share carries no weight and accrues rights. In the nominal profile the targets stand in the ratio 120 (5.7).

$$\vartheta_S = H_{512}(\mathsf{Enc}(\texttt{FW/SHARE/v2},\ \mathit{WorkID})), \qquad \vartheta_B = H_{512}(\mathsf{Enc}(\texttt{FW/SEAL/v2},\ \mathit{WorkID})) \tag{5.5}$$

$$\mathsf{ShareValid} \iff \mathsf{VerifyFW} = 1 \wedge \vartheta_S < \Theta_S, \qquad \mathsf{SealValid} \iff \mathsf{VerifyFW} = 1 \wedge \vartheta_B < \Theta_B \tag{5.6}$$

$$q = \frac{\Theta}{2^{512}}, \qquad \lambda_B = \Lambda q_B, \quad \lambda_S = \Lambda q_S, \qquad T_B = \lambda_B^{-1} = 120 \text{ s}, \quad \lambda_S = 1 \text{ s}^{-1}, \quad \frac{\Theta_S}{\Theta_B} = 120 \tag{5.7}$$

Legend:

- ϑ_S, ϑ_B — the share and seal values in $[0, 2^{512})$; $\Theta_S, \Theta_B \in [1, 2^{512}]$ — the integer targets, fixed by the parent state;
- $\mathsf{VerifyFW}$ — verification of π_{FW} for $(x, \mathit{outputRoot})$;
- q_S, q_B — the per-attempt success probabilities; Λ — the effective attempt rate;
- λ_B, λ_S — the seal and share rates; T_B — the mean block interval.

5.4. Anti-Grinding Theorem

Every lottery value and identifier is a deterministic function of x, and free witness material enters none of them (5.8)–(5.9).

Theorem 5.1 (Anti-grinding). For every context x and any two valid witnesses w, w' of the same statement, (5.8) holds. Each distinct evaluated context therefore yields exactly one draw per lottery (5.9), and post-computation grinding is impossible.

Proof. *seed* depends on x by (5.2), and (X, Y) depends on *seed* by (5.3). *outputRoot*, *WorkID*, *BlockID* and $\vartheta_{B,S}$ depend on (x, XY) by (5.4)–(5.5). The witness does not occur in these maps. ■

$$\Phi(x, w) = \Phi(x, w'), \qquad \Phi = (\mathit{WorkID},\ \mathit{BlockID},\ \vartheta_B,\ \vartheta_S) \tag{5.8}$$

$$\#\{\text{draws per lottery}\} = \#\{\text{distinct } x \text{ evaluated by } \mathsf{Eval}_{\mathrm{FW}}\} \tag{5.9}$$

Legend:

- w, w' — free witness material: signature bytes, prover randomness, Merkle-path order, proof serialization;
- Φ — the tuple of identifiers and lottery values;
- $\mathsf{Eval}_{\mathrm{FW}}$ — the canonical evaluation (5.2)–(5.4); the residual cost of an attempt is governed by A1.

5.5. Fork Choice

Only seals weigh history (5.10). Heaviest-chain selection follows [26]–[28]. Difficulty and windows follow the variable-power FruitChains rules [2], with the fixed target ratio of (5.7). Shares, commercial results, fees and bonds carry zero chain weight (5.11).

$$d_B(\mathsf{blk}) = \frac{2^{512}}{\Theta_B(\mathsf{blk})}, \qquad \mathcal{W}(\mathcal{C}) = \sum_{\mathsf{blk} \in \mathcal{C}} d_B(\mathsf{blk}), \qquad \mathcal{C}^* = \arg \max_{\mathcal{C} \text{ fully validated}} \mathcal{W}(\mathcal{C}) \tag{5.10}$$

$$d_S = \frac{2^{512}}{\Theta_S}, \qquad \omega(\text{seal}) = d_B, \qquad \omega(\text{share}) = \omega(\text{result}) = \omega(\text{fee}) = \omega(\text{bond}) = 0 \tag{5.11}$$

Legend:

- $d_B(\mathsf{blk})$ — the seal weight of block blk; $\mathcal{W}(\mathcal{C})$ — the cumulative chain weight;
- $\mathcal{C}^*$ — the selected branch; on a tie the current valid tip is kept;
- d_S — the share weight, used only for rewards and rights; ω — the contribution of an object to chain weight.

5.6. Delay and Reorganization Bound

The rare seal keeps the fork overlap small (5.12). At constant rates, Theorem 1 of [3] bounds reorganizations under A2–A3 (5.13)–(5.14), and the sufficient depths in the nominal profile follow from (5.15) [29]–[32].

$$P_{\text{near}} = 1 - e^{-\Delta_{\text{eff}}/T_B} \approx 1.653\% \quad (\Delta_{\text{eff}} = 2 \text{ s}), \qquad T_B \geq 60\,\Delta_{\text{eff}} \tag{5.12}$$

$$u = (1-\alpha)\,e^{-\Delta_{\text{eff}}/T_B} > \tfrac{1}{2} \iff \alpha < 1 - \tfrac{1}{2}e^{\Delta_{\text{eff}}/T_B} \approx 0.4916 \tag{5.13}$$

$$P_{\text{reorg}}(k) \leq \min\left\{1,\ \left(2 + 2\sqrt{\frac{u}{1-u}}\right)\left[4u(1-u)\right]^k\right\} \tag{5.14}$$

$$k^* = \left\lceil \frac{\ln\!\left((2 + 2\sqrt{u/(1-u)})/\varepsilon_{\text{set}}\right)}{-\ln\!\left(4u(1-u)\right)} \right\rceil, \qquad k^*(0.10) = 34, \quad k^*(0.25) = 115, \quad k^*(0.40) = 886 \tag{5.15}$$

Legend:

- Δ_{eff} — the bound on time from discovery to full validation; $T_B = 120$ s;
- P_{near} — the probability of overlapping discoveries; α — the adversarial share of effective seal capacity;
- u — the delay-discounted honest share; k — the confirmation depth; $P_{\text{reorg}}(k)$ — the reorganization probability;
- $k^*(\alpha)$ — the sufficient depth for $\varepsilon_{\text{set}} = 2^{-40}$ and $\Delta_{\text{eff}} = 2$ s, that is 68, 230 and 1,772 minutes.

5.7. Rewards and Deviation Bound

Rewards are proportional to mature share weight. The block author receives no leader premium and captures no fees, and rounding remainders stay in the reserve (5.16). On the fairness event, deviation gains at most $\delta_f E_e$ (5.17) [1], [2], [4], [33].

$$w_{i,e} = \sum_{s \in \mathcal{S}_e,\ \text{owner}(s)=i} d_S(s), \qquad \text{Reward}_{i,e} = \left\lfloor E_e \,\frac{w_{i,e}}{\sum_j w_{j,e}} \right\rfloor \tag{5.16}$$

Theorem 5.2 (Deviation bound). On $\mathcal{F}_\delta$ (A5), a coalition of share α satisfies (5.17).

Proof. The honest remainder receives at least $(1-\delta_f)(1-\alpha)E_e$, which gives the first bound. Honest play yields at least $(1-\delta_f)\alpha E_e$. Subtracting the two gives the result. ■

$$P_A^{\text{dev}} \leq [\alpha + \delta_f(1-\alpha)]E_e, \qquad P_A^{\text{hon}} \geq (1-\delta_f)\,\alpha\,E_e, \qquad P_A^{\text{dev}} - P_A^{\text{hon}} \leq \delta_f E_e \tag{5.17}$$

Legend:

- $\mathcal{S}_e$ — the mature qualifying shares of window e; owner(s) — the beneficiary bound in x;
- $w_{i,e}$ — the mature weight of participant i; E_e — the reward budget (7.25);
- δ_f — the fairness tolerance; $P_A^{\text{dev}}, P_A^{\text{hon}}$ — the coalition payout when deviating and when honest.

6. Module 4 — Deterministic Scheduler and D'Hondt Allocation

6.1. ShareID-Indexed Weights

The unit of execution entitlement is a discrete, immutable ShareID of verified mature work, never an address. Its integer weight is fixed by the consensus target of its parent (6.1). Owners pledge bonds for concurrent leases, and the batch size is bounded by budget, readiness and capacity (6.2).

$$w_s = \left\lfloor \frac{2^{512+128}}{\Theta_S(s)} \right\rfloor = \lfloor 2^{128}\, d_S(s) \rfloor, \qquad W = \sum_{s \in \mathcal{S}} w_s, \qquad \frac{2^{128} d_S(s) - w_s}{2^{128} d_S(s)} < 2^{-128} \tag{6.1}$$

$$B_i \geq K_i\, b_{\text{lease}}, \qquad N = \min\Big\{ N_e^{\text{svc}},\ N_{\text{ready}},\ \sum_{i \in \mathcal{O}(\mathcal{S})} K_i^{\text{free}} \Big\}, \qquad N_e^{\text{svc}} \leq \theta\, \frac{W_e}{c_{\text{ref}}} \tag{6.2}$$

Legend:

- s — a source, identified by its *ShareID*; $\mathcal{S}$ — the snapshot of mature qualifying sources; $\Theta_S(s)$ — its consensus target;
- w_s — the integer weight; W — the total weight; $d_S(s)$ — the share weight (5.11);
- B_i — the bond of owner i; K_i — its concurrent-lease capacity; b_{lease} — the bond per lease;
- N — the feasible slots of a batch; N_e^{svc} — the service budget; N_{ready} — the ready tiles;
- $\mathcal{O}(\mathcal{S})$ — the owners holding a source in $\mathcal{S}$; K_i^{free} — the remaining capacity;
- W_e — the mature share weight of epoch e; c_{ref} — the reference weight per service unit; θ — a deployment factor.

6.2. Floor-Quota Defect

Floor quotas lose up to $|\mathcal{S}| - 1$ slots and can assign none when mining power is fragmented (6.3). A per-wallet largest-remainder rule would assign all slots but would be manipulable by moving weight between addresses. Both rules are replaced by (6.4).

$$N - \sum_{s \in \mathcal{S}} \left\lfloor \frac{N w_s}{W} \right\rfloor < |\mathcal{S}|, \qquad w_1 = \cdots = w_9,\ N = 8 \ \Rightarrow\ \left\lfloor \frac{8 w_s}{W} \right\rfloor = 0 \ \ \forall s \tag{6.3}$$

Legend:

- $|\mathcal{S}|$ — the number of sources; the left-hand side of (6.3) is the number of slots a floor rule leaves unassigned;
- the example — nine equal sources and eight slots, where every floor quota is zero.

6.3. Sequential D'Hondt Recursion

Each slot is assigned to the eligible ShareID with the largest D'Hondt quotient $w_s/(a_s + 1)$, with ties broken by the smallest ShareID (6.4). Quotients are compared exactly (6.5). The first batch of a cohort admits the floor-plus-remainder shortcut (6.6). When owner limits bind, the recursion (6.4) is normative.

$$\begin{aligned} s_t^* &= \arg\max_{s \in \mathcal{E}_t} \frac{w_s}{a_s^{(t)} + 1} \quad \text{(ties: least } \mathit{ShareID}_s\text{)}, \qquad \mathcal{E}_t = \{ s \in \mathcal{S} : K^{(t)}_{\text{owner}(s)} > 0 \} \\ a_s^{(t+1)} &= a_s^{(t)} + \mathbb{1}[s = s_t^*], \qquad K_i^{(t+1)} = K_i^{(t)} - \mathbb{1}[i = \text{owner}(s_t^*)] \\ \mathsf{Lease}_t &= (\mathit{tile}_t,\ \mathit{leaseVersion},\ \text{owner}(s_t^*)), \qquad t = 0, 1, \ldots \ \text{ while } \ t < N \ \wedge \ \mathcal{E}_t \neq \emptyset \end{aligned} \tag{6.4}$$

$$\frac{w_s}{a_s + 1} > \frac{w_r}{a_r + 1} \iff w_s\,(a_r + 1) > w_r\,(a_s + 1) \tag{6.5}$$

$$b_s = \left\lfloor \frac{N w_s}{W} \right\rfloor, \qquad N_{\text{rem}} = N - \sum_{s \in \mathcal{S}} b_s < |\mathcal{S}|, \qquad a_s^{(0)} = b_s \text{ then } N_{\text{rem}} \text{ steps of (6.4)} \tag{6.6}$$

Legend:

- t — the slot index; s_t^* — the source served at step t; $\mathcal{E}_t$ — the eligible sources;
- $a_s^{(t)}$ — the slots held by s, persisting across the batches of a cohort; $K_i^{(t)}$ — the remaining capacity of owner i;
- $\mathit{ShareID}_s$ — compared as a canonical 64-byte string, fixed by bound work and not selectable after computation;
- Lease_t — the unique lease created atomically at step t; tile_t — the t-th canonical ready tile;

- r — any other source; b_s — the floor quota; N_{rem} — the remaining slots.

6.4. Completeness and Proportionality

The recursion assigns every feasible slot (6.7). Without binding capacity it selects the N_c smallest D'Hondt markers [53], which gives two-sided bounds and convergence to exact proportionality (6.8)–(6.9).

Theorem 6.1 (Completeness). Each step of (6.4) creates exactly one lease and increments exactly one counter, and $\mathcal{E}_t \neq \emptyset$ for every $t < N$ by (6.2); a batch therefore allocates exactly its N feasible slots (6.7).

Theorem 6.2 (Proportionality). For a fixed cohort without binding capacity, after N_c slots, (6.8) and (6.9) hold.

Proof. The selected markers $\nu_{s,j} = j/w_s$ are the N_c smallest. At most N_c markers do not exceed N_c/W, which gives the lower bound. Let ν^* be the N_c-th selected marker. Then $a_r \geq \nu^* w_r - 1$ for every r, so $N_c \geq \nu^* W - |\mathcal{S}|$; combined with $a_s \leq \nu^* w_s$ this gives the upper bound. Dividing by N_c yields (6.9). ■

$$\sum_{s\in\mathcal{S}} (a_s^{(N)} - a_s^{(0)}) = N \leq \sum_{i\in\mathcal{O}(\mathcal{S})} K_i^{\text{free}} \qquad (6.7)$$

$$\left\lfloor \frac{N_c\, w_s}{W} \right\rfloor \leq a_s(N_c) \leq \frac{(N_c + |\mathcal{S}|)\, w_s}{W} \qquad (6.8)$$

$$\lim_{N_c\to\infty} \frac{a_s(N_c)}{N_c} = \frac{w_s}{W} \qquad (6.9)$$

Legend:

- N_c — the cumulative slots allocated in the cohort; $a_s(N_c)$ — those held by s;
- $\nu_{s,j}$ — the j-th marker of s; ν^* — the last selected marker;
- $a_s^{(0)}, a_s^{(N)}$ — the counter of s before and after the batch; (6.7) — no feasible slot is left unassigned.

6.5. Aggregation Invariance

The allocation is computed over discrete ShareIDs, and owner labels enter (6.4) only through capacity. An entity's allocation is therefore the sum over its ShareIDs, independent of how they are grouped into owners or addresses (6.10). D'Hondt's large-unit bias cannot be captured by aggregation: the largest allocatable unit is a single share, and its weight depends only on the consensus target of its parent, not on holdings (6.11). Merging identities yields no premium over the best split with the same bonded capacity (6.10), (6.12).

Theorem 6.3 (No aggregation premium). Let an entity E hold $\mathcal{S}_E \subseteq \mathcal{S}$.

1. Without binding capacity, (6.10) holds for any two partitions P, P' of $\mathcal{S}_E$ into owners, and $\Pi_{\text{agg}} = 0$.
2. With binding capacity, a merged owner of capacity K_E is reproduced exactly by a partition whose capacities sum to K_E. Hence $\Pi_{\text{agg}} \leq 0$, and (6.12) holds.

Proof. (1) Without binding capacity, (6.4) selects the N_c smallest elements of the marker multiset $\{(j/w_s, \mathit{ShareID}_s)\}$, which carries no owner label. Summing (6.8) over $\mathcal{S}_E$ gives (6.11). (2) Give each part the capacity it receives in the merged run. By induction on t, the split run selects the same s_t^*, because a part becomes ineligible only after it has received its final merged allocation. (6.12) follows from $K_i^{(t)} \geq 0$. ■

$$a_E^P(N_c) = a_E^{P'}(N_c) = \sum_{s\in\mathcal{S}_E} a_s(N_c), \qquad \Pi_{\text{agg}} := a_E^{\text{merged}} - \max_{P:\ \sum_k K_k = K_E} a_E^P \leq 0 \qquad (6.10)$$

$$w_s = \omega_\Theta(\Theta_S(\text{parent}(s))), \qquad \left\lfloor \frac{N_c W_E}{W} \right\rfloor - |\mathcal{S}_E| < a_E(N_c) \leq \frac{(N_c + |\mathcal{S}|)\, W_E}{W} \qquad (6.11)$$

$$a_E \leq K_E = \sum_{i\in E} K_i \qquad (6.12)$$

Legend:

- E — an entity, for example a whale controlling many keys; $\mathcal{S}_E$ — its ShareIDs; $W_E = \sum_{s\in\mathcal{S}_E} w_s$;
- P, P' — partitions of $\mathcal{S}_E$ into owners or addresses; a_E^P — the slots of E under P; K_k — the capacity of part k;
- Π_{agg} — the aggregation premium of one merged owner over the best capacity-matched split; it is exactly 0 without binding capacity;
- ω_Θ — the weight map of (6.1); parent(s) — the parent block under which s was found;
- (6.11) — the entity bounds without binding capacity; K_E — the total bonded capacity of E.

6.6. Leases and Phase Machine

Each admitted tile has exactly one active lease bound to one executor, and the bond reserved across an executor's leases never exceeds its bond (6.13). The job lifecycle is the transition relation (6.14). Deadlines separate client and executor responsibility (6.15).

$$\begin{gathered}\mathsf{Lease} = (\mathit{jobID},\ \mathit{tileID},\ \mathit{leaseVersion},\ \mathit{executor},\ \mathit{escrowID},\ \mathit{price},\ B_{\text{res}},\ h_{\text{deadline}})\\ \big|\text{active}(\mathit{tileID})\big| \le 1, \qquad \sum_{\ell\in\text{active}(i)} B_{\text{res},\ell} \le B_i\end{gathered} \tag{6.13}$$

$$\begin{gathered}\bot \xrightarrow{\texttt{ESCROW_FUND_AND_QUEUE}} \text{Funded},\\ \text{Funded} \xrightarrow{\texttt{JOB_ADMIT_AND_LEASE}} \text{Admitted}, \qquad \text{Funded} \xrightarrow{\text{TTL}\vee\text{cancel}} \text{Refunded},\\ \text{Admitted} \xrightarrow{\texttt{INPUT_READY},\ h\le h_{\text{admit}}+D_{\text{avail}}} \text{Running}, \qquad \text{Admitted} \xrightarrow{h>h_{\text{admit}}+D_{\text{avail}}} \text{Retry},\\ \text{Running} \xrightarrow{\texttt{RESULT_SETTLE}} \text{Settled}, \qquad \text{Running} \xrightarrow{\texttt{LEASE_TIMEOUT},\ h>h_{\text{deadline}}} \text{Retry},\\ \text{Retry} \xrightarrow{\texttt{LEASE_RETRY},\ \beta_{\text{retry}}>0} \text{Admitted}, \qquad \text{Retry} \xrightarrow{\texttt{ESCROW_REFUND},\ \beta_{\text{retry}}=0} \text{Refunded}\end{gathered} \tag{6.14}$$

$$h_{\text{ready}} \le h_{\text{admit}}+D_{\text{avail}}, \qquad h_{\text{deadline}} = h_{\text{ready}}+\text{TTL}_{\text{comp}}, \qquad \text{penalty}_{\text{executor}}(h > h_{\text{admit}}+D_{\text{avail}}) = 0 \tag{6.15}$$

Legend:

- Lease — the lease tuple; B_{res} — its bond reservation; active($\cdot$) — the active leases of a tile or of executor i;
- $\bot$ — the empty job; Funded, Admitted, Running, Retry — working states; Settled, Refunded — terminal states;
- h — the canonical height; h_{admit} — the height of `JOB_ADMIT_AND_LEASE`; D_{avail} — the data window;
- h_{ready} — the height of `INPUT_READY`; TTL_{comp} — the computation TTL; h_{deadline} — the delivery deadline;
- β_{retry} — the remaining retry budget authorized at funding; each retry is a paid re-admission with a new *leaseVersion*.

6.7. Ordering and Throughput

For a fixed canonical state the allocation is unique, and payment is bound to the lessee, which eliminates the execution race and priority auctions [34] (6.16). Queue inclusion rests on A3. Throughput is bounded by the narrowest resource (6.17).

$$\mathsf{Alloc}(\Sigma) \text{ is a function of } \Sigma, \qquad \mathsf{Pay}(e, \mathit{tileID}) > 0 \ \Rightarrow\ e = \mathsf{Lease}(\mathit{tileID}).\mathit{executor} \tag{6.16}$$

$$T_{\text{compute}} \approx \left\lceil \frac{M_T}{N_G} \right\rceil \tau_C, \qquad \text{Throughput} \le \min\Big\{\sum_i \mu_i,\ \frac{B_{\text{DAL}}}{s_C},\ \min_v \frac{B_v}{s_C},\ \min_v \frac{\text{Compute}_v}{v_C}\Big\} \tag{6.17}$$

Legend:

- Alloc — the scheduler output; $\mathsf{Pay}(e, \mathit{tileID})$ — the payment to party e for a tile;
- T_{compute} — the ideal job time; M_T — the equal tiles; N_G — the executors; τ_C — the time per tile;
- μ_i — the rate of executor i; B_{DAL}, B_v — the bandwidth of P-DAL and of node v; Compute_v — the validation capacity of v;
- s_C, v_C — the data size and the validation cost per unit of work.

7. Module 5 — Hybrid State, zk-TWL and Tokenomics

7.1. State and Operation Set

The state is a twelve-component tuple over a single native asset, and the operation set is closed: there is no EVM, no user contract and no user-issued token (7.1)–(7.2).

$$\Sigma = (\mathcal{U},\ \mathcal{T}_{\text{note}},\ \mathcal{N},\ S_z,\ \mathcal{E},\ \mathcal{B},\ \mathcal{L},\ \mathcal{Q},\ \mathcal{V},\ R_a,\ R_l,\ P_{\text{pending}}), \qquad \mathsf{Assets} = \{\texttt{TKN}\} \tag{7.1}$$

$$\begin{aligned}\mathsf{Ops} = \{\ &\texttt{TWL_TRANSFER},\ \texttt{SHIELD},\ \texttt{ZK_TRANSFER},\ \texttt{UNSHIELD},\\ &\texttt{ESCROW_FUND_AND_QUEUE},\ \texttt{JOB_ADMIT_AND_LEASE},\ \texttt{INPUT_READY},\\ &\texttt{RESULT_SETTLE},\ \texttt{LEASE_TIMEOUT},\ \texttt{LEASE_RETRY},\ \texttt{ESCROW_REFUND},\\ &\texttt{BOND_LOCK},\ \texttt{BOND_RELEASE},\ \texttt{BOND_SLASH},\ \texttt{FW_REWARD},\ \texttt{RESERVE_MATURE},\\ &\texttt{L2_DEPOSIT},\ \texttt{L2_BATCH},\ \texttt{L2_WITHDRAW},\ \texttt{L2_EXIT}\ \}\end{aligned} \tag{7.2}$$

Legend:

- $\mathcal{U}$ — public UTXOs; $\mathcal{T}_{\text{note}}$ — the append-only note-commitment tree; $\mathcal{N}$ — the spent nullifiers; S_z — the shielded pool balance;
- $\mathcal{E}, \mathcal{B}$ — escrow and bond accounts; $\mathcal{L}, \mathcal{Q}$ — leases with scheduler counters, and the queue; $\mathcal{V}$ — L2 vaults;
- $R_a, R_l, P_{\text{pending}}$ — the available reserve, the locked reserve and the segregated reward payables;
- Ops — the closed operation set; no operation type confers privilege.

7.2. TWL Authorization

A public spend is valid only if each presented key matches the locking policy of the consumed output read from state, (7.3). The authorization message commits to the complete effect digest, so no weak signature-hash mode exists (7.4)–(7.5). Conflicts are local, which admits parallel validation (7.6) [26].

$$\forall\, \mathit{op} \in \mathit{inputOutpoints}:\ \mathsf{Match}(\mathit{pk}_{\mathit{op}},\ \mathit{lockingPolicy}(\mathit{op})) = 1 \tag{7.3}$$

$$\begin{aligned}\mathit{TxCore} = \mathsf{Enc}(&\mathit{chainID},\ \mathit{genesisID},\ \mathit{protocolVersion},\ \mathit{operationType},\\ &\mathit{inputOutpoints},\ \mathit{allOutputsWithAmountsAndPolicies},\ \mathit{publicPoolDelta},\\ &\mathit{nullifiers},\ \mathit{acceptedAnchors},\ \mathit{fee},\ \mathit{escrowID},\ \mathit{jobManifestHash},\\ &\mathit{expectedAccountVersion},\ \mathit{leaseID},\ \mathit{leaseVersion},\ \mathit{deadline})\end{aligned} \tag{7.4}$$

$$\mathit{TxID} = H_{512}(\mathsf{Enc}(\texttt{TX/EFFECTS/v2},\ \mathit{TxCore})), \qquad \mathit{AuthMsg} = \mathsf{Enc}(\texttt{PoUW/TX/v2},\ \mathit{TxID}) \tag{7.5}$$

$$\begin{aligned}\mathsf{Conflict}(\mathit{tx}, \mathit{tx}') \iff &\text{ shared outpoint } \vee \text{ shared nullifier}\\ &\vee \text{ shared } (\mathit{account}, \mathit{expectedVersion}) \vee \text{ shared } \mathit{receiptID}\end{aligned} \tag{7.6}$$

Legend:

- op — a consumed outpoint; $\mathit{pk}_{\mathit{op}}$ — the presented key; $\mathit{lockingPolicy}(\mathit{op})$ — the immutable policy of the output;

- *TxCore* — the sixteen-field effect digest; *TxID* — the effect identifier; *AuthMsg* — the message signed under (2.3);
- Conflict — the only relation that serializes validation; ownership changes only by authorized spend and new output.

7.3. zk-TWL Notes

Note ownership is knowledge of a 512-bit secret bound by hash relations that a post-quantum zero-knowledge proof verifies (A4) [5], [6], [36]. The relation contains no discrete-logarithm commitment, signature or pairing (7.7)–(7.10). The nullifier depends only on the note and its secret, so each note has exactly one spend tag.

$$pk_z = H_{512}(\mathsf{Enc}(\texttt{ZK/SPEND/v2},\ \mathit{chainID},\ s)), \qquad nk = \mathsf{KMAC256}(s,\ \mathit{chainID},\ 512,\ \texttt{PoUW/NK/v2}) \tag{7.7}$$

$$\begin{aligned} cm &= H_{512}(\mathsf{Enc}(\texttt{NOTE/v2},\ \mathit{chainID},\ \texttt{TKN},\ v,\ pk_z,\ \rho,\ r)) \\ nf &= \mathsf{KMAC256}(nk,\ \mathsf{Enc}(\mathit{chainID},\ cm),\ 512,\ \texttt{PoUW/NF/v2}) \end{aligned} \tag{7.8}$$

$$\begin{aligned} &\mathcal{R}_{\mathrm{zk}} = \{(\mathsf{stmt};\ \mathsf{wit}) : cm \in \mathsf{Tree}(\mathit{anchor}) \ \wedge\ (7.7) \ \wedge\ (7.8) \ \wedge\ (7.10) \ \wedge\ 0 \le v \le M\} \\ &\mathsf{stmt} = (\mathit{anchor},\ \{nf\},\ \{cm'\},\ \Delta_{\mathrm{public}},\ f,\ \mathit{TxCore}), \qquad \mathsf{wit} = (s,\ v,\ \rho,\ r,\ \mathit{path},\ \text{openings of } cm') \end{aligned} \tag{7.9}$$

$$\sum v_{\mathrm{in}} = \sum v_{\mathrm{out}} + \Delta_{\mathrm{public}} + f, \qquad nf \notin \mathcal{N}, \qquad cm' \notin \mathcal{T}_{\mathrm{note}} \tag{7.10}$$

Legend:

- s — the 512-bit spending secret; ρ, r — independent 512-bit openings; v — the note value in atoms;
- pk_z, nk — the spending key and the nullifier key; $\mathsf{KMAC256}(K, X, L, S)$ — key, input, output bits, customization;
- cm, nf — the note commitment and nullifier; cm' — the new commitments;
- $\mathcal{R}_{\mathrm{zk}}$ — the relation proven in zero knowledge; $\mathsf{stmt}, \mathsf{wit}$ — its public statement and secret witness;
- *anchor* — an accepted root within the anchor window; $\mathsf{Tree}(\mathit{anchor})$ — the committed notes under it; *path* — the Merkle path;
- Δ_{public} — the public pool delta (positive when value leaves the pool); f — the fee; M — the supply.

7.4. Private Funding and Disclosure Set

A ZK-funded escrow consumes only shielded notes and has no public co-input or change, so the link to its source notes is hidden (7.11). The manifest carries raw matrices and terms only, which decouples the task from its business context. The public and hidden sets are explicit (7.12).

$$\sum_i v_i^{\mathrm{in}} = \sum_j v_j^{\mathrm{change}} + D + f_{\mathrm{queue}}, \qquad \Delta_{\mathrm{public}} = D \tag{7.11}$$

$$\begin{aligned} \mathcal{D}_{\mathrm{pub}} &= \{\text{budget},\ \text{timing},\ \text{class},\ \text{dimensions},\ \text{matrices in P-DAL}\} \\ \mathcal{D}_{\mathrm{hidden}} &= \{\text{source notes},\ \text{payer identity},\ \text{business context}\} \end{aligned} \tag{7.12}$$

Legend:

- $v_i^{\mathrm{in}}, v_j^{\mathrm{change}}$ — the values of consumed and change notes; D — the escrow deposit; f_{queue} — the queue fee (7.20);
- $\mathcal{D}_{\mathrm{pub}}$ — data public by design, including matrices reconstructed by consensus nodes;
- $\mathcal{D}_{\mathrm{hidden}}$ — data protected by zk-TWL and by the manifest; envelopes follow (2.6).

7.5. Atomic Transitions and Settlement

Every operation moves value between disjoint categories by deltas that sum to zero (7.13), and it commits as one database transaction or not at all (7.14). A bridge is therefore a single transition with no extra token, multisig or bridge consensus: the inputs are consumed and the escrow is credited in one indivisible commit. Settlement is the guarded transition (7.15), and the lifecycle deltas are (7.16). A worked budget is (7.17).

$$\Delta U + \Delta S_z + \Delta E_{\text{esc}} + \Delta B_{\text{bond}} + \Delta V_{\text{L2}} + \Delta R_a + \Delta R_l + \Delta P_{\text{pending}} = 0 \tag{7.13}$$

$$\Sigma' = \begin{cases} \Sigma \oplus \Delta & \text{if } \mathsf{Pre}(\Sigma, tx) \wedge \mathsf{Inv}(\Sigma \oplus \Delta), \\ \Sigma & \text{otherwise} \end{cases} \tag{7.14}$$

$$\begin{aligned} \mathsf{Pre}_{\text{settle}} \iff \mathsf{LeaseMatch}(&\mathit{jobID},\ \mathit{tileID},\ \mathit{leaseVersion},\ \mathit{executor}) \ \wedge\ \mathit{status} = \text{Running} \\ &\wedge\ h \le h_{\text{deadline}} \ \wedge\ \mathit{receiptID} \notin \mathcal{R} \ \wedge\ \text{ComputeValid} \ \wedge\ \mathsf{Admit}_{\text{DA}} \\ &\wedge\ E_{\text{esc}} \ge v_{\text{exec}} + v_{\text{DA}} + v_{\text{ref}}, \qquad \mathcal{R}' = \mathcal{R} \cup \{\mathit{receiptID}\} \end{aligned} \tag{7.15}$$

$$\begin{aligned} \texttt{ESCROW_FUND_AND_QUEUE}: &\quad \Delta U \text{ or } \Delta S_z = -(D + f_{\text{queue}}), \quad \Delta E_{\text{esc}} = D, \quad \Delta R_l = f_{\text{queue}} \\ \texttt{JOB_ADMIT_AND_LEASE}: &\quad \Delta E_{\text{esc}} = -F^{\text{base}}, \quad \Delta R_l = F^{\text{base}} \\ \texttt{RESULT_SETTLE}: &\quad \Delta E_{\text{esc}} = -(v_{\text{exec}} + v_{\text{DA}} + v_{\text{ref}}), \quad \Delta U + \Delta S_z = v_{\text{exec}} + v_{\text{DA}} + v_{\text{ref}} \\ \texttt{ESCROW_REFUND}: &\quad \Delta E_{\text{esc}} = -v_{\text{ref}}, \quad \Delta U + \Delta S_z = v_{\text{ref}} \\ \texttt{BOND_SLASH}: &\quad \Delta B_{\text{bond}} = -v_{\text{slash}}, \quad \Delta R_l = v_{\text{slash}} \\ \texttt{RESERVE_MATURE}: &\quad \Delta R_l = -v_{\text{mat}}, \quad \Delta R_a = v_{\text{mat}} \\ \text{epoch budget}: &\quad \Delta R_a = -v_{\text{seg}}, \quad \Delta P_{\text{pending}} = v_{\text{seg}} = \textstyle\sum_i \text{Reward}_{i,e} \le E_e \\ \texttt{FW_REWARD}: &\quad \Delta P_{\text{pending}} = -v_{\text{pay}}, \quad \Delta U = v_{\text{pay}} \end{aligned} \tag{7.16}$$

$$D = F^{\text{base}} + v_{\text{exec}} + v_{\text{DA}} + v_{\text{ref}} = (10 + 80 + 5 + 5)\ \text{TKN} = 100\ \text{TKN} \tag{7.17}$$

Legend:

- Δ — the change of a balance category in one operation; $U, S_z, E_{\text{esc}}, B_{\text{bond}}, V_{\text{L2}}, R_a, R_l, P_{\text{pending}}$ — the categories of (7.19);
- $\Sigma \oplus \Delta$ — the candidate state; Pre — the authorization, range, unspent, uniqueness and coverage checks; Inv — the invariants (7.19) and non-negativity;
- $\mathsf{LeaseMatch}$ — agreement with the active lease; $\mathcal{R}$ — the used receipt identifiers; $\mathsf{Admit}_{\text{DA}}$ — the reconstruction rule (4.5);
- D — the escrow budget; $f_{\text{queue}}, F^{\text{base}}$ — the queue fee and the Base Fee;
- $v_{\text{exec}}, v_{\text{DA}}, v_{\text{ref}}, v_{\text{slash}}, v_{\text{mat}}, v_{\text{seg}}, v_{\text{pay}}$ — the executor, DA, refund, slash, maturing, segregated and paid amounts; rounding remainders of (5.16) stay in R_a;
- a reorganization reverts all deltas jointly, including nullifiers, receipts and scheduler counters.

7.6. Supply Invariant

The supply is fixed at M atoms (7.18). Genesis allocates 10% to the team and 90% to the Network Emission Reserve, and the sum of the eight disjoint balances equals M in every reachable state (7.19).

Theorem 7.1 (Supply conservation). If Genesis satisfies (7.19) and every accepted operation satisfies (7.13) under the commit rule (7.14), then every reachable state satisfies (7.19).

Proof. By induction on accepted operations. A rejected operation leaves Σ unchanged, and a reorganization reverts complete delta sets. ■

$$\begin{aligned} &1\ \text{TKN} = 10^8\ \text{atoms}, \qquad M = 1\,500\,000\,000\,000\,000\ \text{atoms} < 2^{51} < p \\ &U^{(0)}_{\text{team}} = 1\,500\,000\ \text{TKN}, \qquad R^{(0)}_a = 13\,500\,000\ \text{TKN} \end{aligned} \tag{7.18}$$

$$\boxed{U + S_z + E_{\text{esc}} + B_{\text{bond}} + V_{\text{L2}} + R_a + R_l + P_{\text{pending}} = M}, \qquad \sum_{\text{unspent notes}} v_i = S_z \tag{7.19}$$

Legend:

- M — the fixed supply in atoms; $U^{(0)}_{\text{team}}, R^{(0)}_a$ — the Genesis team UTXOs and available reserve;
- p — the base modulus; bounded sums balance in $\mathbb{F}_p$ only after range proofs, and larger sums use limbs with carries;
- the categories of (7.19) — disjoint and non-negative; every atom is counted exactly once.

7.7. Fee Schedule and Anti-Spam Base Fee

Queue entry and reserved capacity are priced separately, with positive integer tariffs and no tips or ordering auctions (7.20) [35]. At the exact moment of `JOB_ADMIT_AND_LEASE`, the Base Fee transfers irretrievably from the escrow to the locked reserve (7.21). If the client does not provide the data matrix by $h_{\text{admit}} + D_{\text{avail}}$, it forfeits the queue fee and the Base Fee, and the executor bears no penalty (7.22). Queue capacity therefore cannot be held for free.

$$F_J = f_J^{\text{queue}} + F_J^{\text{base}} + F_J^{\text{exec}} + F_J^{\text{DA}}, \qquad f_{\text{queue}} = f_{\text{entry}} + \tau_{\text{byte}}\, B_{\text{queue}}\, H_{\text{TTL}} > 0 \tag{7.20}$$

$$\begin{aligned} \texttt{JOB_ADMIT_AND_LEASE}: \;& E_J \leftarrow E_J - F_J^{\text{base}}, \qquad R_l[e(h_{\text{admit}}) + L] \leftarrow R_l[e(h_{\text{admit}}) + L] + F_J^{\text{base}} \\ & \forall\, op \in \mathsf{Ops}: \;\; \text{flow}_{op}(R_l \to E_J) = 0 \end{aligned} \tag{7.21}$$

$$\begin{aligned} \neg\, \texttt{INPUT_READY} \text{ by } h_{\text{admit}} + D_{\text{avail}} \;\Rightarrow\; & \text{Loss}_{\text{client}} = f_J^{\text{queue}} + F_J^{\text{base}}, \\ & \text{penalty}_{\text{executor}} = 0, \qquad F_J^{\text{exec}} + F_J^{\text{DA}} \in E_J \end{aligned} \tag{7.22}$$

$$\big|\{op : op.id = \iota\}\big| \le 1 \quad \text{for } \iota \in \{\mathit{ShareID},\ \mathit{admissionID},\ \mathit{receiptID},\ \mathit{rewardID}\} \tag{7.23}$$

Legend:

- F_J — the total charge of job J; $F_J^{\text{exec}}, F_J^{\text{DA}}$ — the executor and DA remuneration, held in escrow until earned;
- f_{entry} — the entry and validation fee; τ_{byte} — the price per byte-block; B_{queue} — the accounted job-state bytes; H_{TTL} — the paid retention in blocks;
- E_J — the escrow of J; $R_l[\cdot]$ — the locked reserve indexed by maturity epoch; $e(h)$ — the epoch containing height h; L — the maturation delay in epochs;
- $\text{flow}_{op}(R_l \to E_J)$ — value moved by op from the locked reserve back to the escrow or the client, which is zero for every operation;
- $\text{Loss}_{\text{client}}$ — the forfeited amount; the unearned remuneration stays in E_J for a paid re-admission or is refunded (6.14);
- ι — an identifier class; repeated identifiers are rejected without state change.

7.8. Recycling and Reserve Dynamics

Fees are recycled, never burned. They mature from R_l into R_a and fund Fresh Work rewards through a geometric payout rule (7.24)–(7.25). Under a constant inflow the reserve converges to a steady state in which rewards equal the fee inflow, so the reward stream is perpetual (7.26).

$$\text{client} \to R_l \xrightarrow{\text{maturation}} R_a \to \text{Fresh Work rewards} \tag{7.24}$$

$$E_e = \eta\, R_e, \qquad R_{e+1} = (1 - \eta)\, R_e + F_{e-L}, \qquad 0 < \eta < 1 \tag{7.25}$$

$$R^* = \frac{\bar{F}}{\eta}, \qquad E^* = \bar{F}, \qquad R_e - R^* = (1-\eta)^e\,(R_0 - R^*), \qquad \eta = 1 - 2^{-1/H_R} \tag{7.26}$$

Legend:

- R_e — the available reserve in epoch e; E_e — the reward budget; η — the rational payout rate;
- F_{e-L} — the matured fee inflow of epoch $e-L$; $\bar{F}$ — a constant inflow per epoch;
- R^*, E^* — the steady state; $R_0 = 13\,500\,000$ TKN — the Genesis reserve; H_R — the reserve half-life in epochs.

7.9. Self-Dealing Theorem

A coalition that pays itself for fictitious jobs gains neither weight nor shares, and it recovers at most the discounted fraction $\bar{\alpha}\beta$ of each Base Fee (7.27)–(7.28).

Theorem 7.2 (Self-dealing is loss-making). For every coalition with $\bar{\alpha}\beta < 1$, (7.29) holds, even when execution and DA payments are fully recovered and computation is free.

Proof. The coalition pays F at admission and recovers at most $\bar{\alpha}\beta F$ by (7.28). The remaining costs are non-negative. ■

For $\alpha = 25\%$, $\delta_f = 1\%$ and $\beta = 1$: $\bar{\alpha} = 25.75\%$, so the expected loss is at least $0.7425\,F$. Outside the fairness event, $\bar{\alpha}$ is replaced by (7.30).

$$\bar{\alpha} = \alpha + \delta_f(1-\alpha) \tag{7.27}$$

$$PV_{\text{return}} \le \bar{\alpha}F \sum_{t=0}^{\infty} \eta(1-\eta)^t \beta^{L+1+t} = \bar{\alpha}F\,\frac{\eta\,\beta^{L+1}}{1-(1-\eta)\beta} \le \bar{\alpha}\,\beta\,F \tag{7.28}$$

$$\boxed{\mathbb{E}[\Delta\Pi_{\text{self}}] \le -(1-\bar{\alpha}\beta)\,F - C_{\text{res}} - C_{\text{cap}} < 0} \tag{7.29}$$

$$\bar{\alpha}_\varepsilon = \bar{\alpha} + \varepsilon_{\text{bad}}(1-\bar{\alpha}) \tag{7.30}$$

Legend:

- α — the coalition's share of effective attempts; δ_f — the fairness tolerance (A5); $\bar{\alpha}$ — its bound on future payout share;
- F — the Base Fee of the fictitious job; $\beta \in (0,1]$ — the per-epoch discount factor; η, L — as in (7.25);
- PV_{return} — the present value returned to the coalition; $C_{\text{res}}, C_{\text{cap}}$ — the resource and capital costs;
- ε_{bad} — the probability of the complement of $\mathcal{F}_\delta$; invalid traffic that pays no fee is bounded by (7.31).

7.10. Resource Limits and L2 Slots

Every block satisfies a componentwise resource vector that accounts for post-quantum object sizes (7.31). Layer 2 attaches only through versioned settlement slots whose vaults cover all claims (7.32) [7], [37]. Activation requires DA rules and an operator-failure exit.

$$\sum_{tx \in \mathsf{blk}} \vec{c}(tx) \preceq (B_{\text{bytes}},\ B_{\text{sig}},\ B_{\text{ZK}},\ B_{\text{state}},\ B_{\text{DA}}), \qquad |\sigma_{\mathsf{ML\text{-}DSA\text{-}87}}| = 4627\text{ B}, \quad |\sigma_{\mathsf{SLH\text{-}DSA}}| = 29\,792\text{ B} \tag{7.31}$$

$$\begin{aligned} \mathsf{RollupSlot} = (&\mathit{RID},\ \mathit{verifierID},\ \mathit{programHash},\ \mathit{root},\ \mathit{seq},\\ &\mathit{inboxCursor},\ \mathit{outboxRoot},\ \mathit{vaultBalance},\ \mathit{DApolicy},\ \mathit{exitPolicy}) \\ V_{\mathit{RID}} = &\text{ active L2 claims} + \text{authorized unpaid exits} + \text{unbooked deposits} \end{aligned} \tag{7.32}$$

Legend:

- $\vec{c}(tx)$ — the resource vector of tx; $\preceq$ — componentwise inequality;
- $B_{\text{bytes}}, B_{\text{sig}}, B_{\text{ZK}}, B_{\text{state}}, B_{\text{DA}}$ — the block limits on bytes, signature checks, ZK checks, state writes and DA data;
- RID — the slot identifier; V_{RID} — its vault; an L2 program never writes the reserve, foreign vaults or PoUW accounts.

8. Conclusion

8.1. Guarantees

Table 4 lists each guarantee with the result that establishes it and its hypothesis.

Table 4. Guarantees, results and hypotheses.

Property	Result	Hypothesis
Tile correctness	(3.7), (3.14): Freivalds term $\leq p^{-6}$	Result fixed before challenges; A4
128-bit post-quantum proof target	(2.12), (3.13) over $\mathbb{F}_{p^8}$	A4
Quantum-resistant binding	(2.9): $Q_{\text{col}}(512) \approx 2^{170.7}$	A7
No post-computation grinding	Theorem 5.1	Sixteen-field x bound before evaluation
Weight only from seals	(1.1), (5.10)–(5.11)	Full validation (4.5)
Settlement depth	(5.14)–(5.15)	A2, A3
Bounded deviation gain	Theorem 5.2	A5
Complete, proportional allocation	Theorems 6.1–6.2	Fixed cohort
No aggregation premium	Theorem 6.3: $\Pi_{\text{agg}} \leq 0$	Equality without binding capacity; bound (6.12)
One lease per tile, no execution race	(6.13), (6.16)	Canonical state
Availability	(4.4), (4.5)	A6 for sampling; reconstruction for consensus
Supply conservation	Theorem 7.1	Commit rule (7.14)
Irretrievable Base Fee, anti-spam	(7.21)–(7.22)	`JOB_ADMIT_AND_LEASE` semantics
Loss-making self-dealing	Theorem 7.2	$\bar{\alpha}\beta < 1$

8.2. Conformance Invariants

Every conforming implementation satisfies the following invariants:

1. x binds all sixteen fields of (5.1) before evaluation, and Φ is witness-independent (5.8).
2. Only seals contribute to $\mathcal{W}$ (5.11).
3. Each *ShareID*, *admissionID*, *receiptID* and *rewardID* is used at most once (7.23).
4. Allocation follows (6.4) over ShareIDs; (6.7) holds, and owner capacity is enforced.
5. $|\text{active}(tileID)| \leq 1$ (6.13).
6. A missed data window never penalizes the executor (6.15).
7. Consensus nodes admit blocks only under (4.5).
8. Every hidden amount and public delta is bound by $\mathcal{R}_{\text{zk}}$ or *TxCore*; any field mutation invalidates authorization (7.4)–(7.5).
9. Every transition obeys (7.13)–(7.14), and a reorganization reverts complete delta sets.
10. (7.19) holds in every reachable state.
11. The Base Fee moves to R_l only at `JOB_ADMIT_AND_LEASE` and never returns to the escrow (7.21).
12. L2 programs cannot write native PoUW state (7.32).
13. ComputeValid never substitutes for PaymentValid (4.6)–(4.7).
14. Every object carries a $cryptoSuiteID \in \mathcal{K}_{\text{active}}(h)$ (2.2).

8.3. Concluding Statement

PoUW AppChain derives chain weight, reward funding and paid-compute allocation from one resource: verified Fresh Work on the canonical 512×512 tile. Commercial demand is economically productive but never buys consensus weight. The architecture is post-quantum at every layer, conserves a fixed supply exactly, and is specified by the theorems and invariants above.

References

[1] R. Pass and E. Shi, "FruitChains: A Fair Blockchain," Cryptology ePrint Archive, Paper 2016/916, 2016. eprint.iacr.org/2016/916.

[2] X. Wang, V. V. Muppirala, L. Yang, S. Kannan and P. Viswanath, "Securing Parallel-chain Protocols under Variable Mining Power," arXiv:2105.02927, 2021, §5.2 and Appendix E. arxiv.org/abs/2105.02927.

[3] D. Guo and L. Ren, "Bitcoin's Latency–Security Analysis Made Simple," arXiv:2203.06357, 2022, Theorem 1. arxiv.org/abs/2203.06357.

[4] A.-P. Stouka and T. Zacharias, "On the (De)centralization of FruitChains," arXiv:2307.12422, 2023; DOI 10.1109/CSF57540.2023.00020. arxiv.org/abs/2307.12422.

[5] D.-E. Hopwood, S. Bowe, T. Hornby and N. Wilcox, "Zcash Protocol Specification" (accessed 8 October 2026), zips.z.cash/protocol/protocol.pdf; and D.-E. Hopwood, J. Grigg and K. Nuttycombe, "ZIP 315: Best Practices for Wallet Implementations" (status: Draft), zips.z.cash/zip-0315.

[6] U. Haböck and Al Kindi, "A note on adding zero-knowledge to STARKs," Cryptology ePrint Archive, Paper 2024/1037, 2024. eprint.iacr.org/2024/1037.

[7] "Zero-knowledge rollups," Ethereum.org developer documentation. ethereum.org/developers/docs/scaling/zk-rollups.

[8] R. Freivalds, "Probabilistic machines can use less running time," in Information Processing 77, Proceedings of the IFIP Congress, North-Holland, 1977, pp. 839–842.

[9] R. Freivalds, "Fast probabilistic algorithms," in Mathematical Foundations of Computer Science 1979 (MFCS 1979), Lecture Notes in Computer Science, vol. 74, Springer, 1979. doi.org/10.1007/3-540-09526-8_5.

[10] A. Fiat and A. Shamir, "How to Prove Yourself: Practical Solutions to Identification and Signature Problems," in Advances in Cryptology — CRYPTO '86, Lecture Notes in Computer Science, vol. 263, Springer, 1987, pp. 186–194.

[11] E. Ben-Sasson, I. Bentov, Y. Horesh and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," Cryptology ePrint Archive, Paper 2018/046, 2018. eprint.iacr.org/2018/046.

[12] U. Haböck, D. Levit and S. Papini, "Circle STARKs," Cryptology ePrint Archive, Paper 2024/278, 2024. eprint.iacr.org/2024/278.

[13] E. Ben-Sasson, I. Bentov, Y. Horesh and M. Riabzev, "Fast Reed-Solomon Interactive Oracle Proofs of Proximity," in 45th International Colloquium on Automata, Languages, and Programming (ICALP 2018), LIPIcs, vol. 107, Schloss Dagstuhl, 2018, pp. 14:1–14:17. doi.org/10.4230/LIPIcs.ICALP.2018.14.

[14] E. Ben-Sasson, A. Chiesa and N. Spooner, "Interactive Oracle Proofs," Cryptology ePrint Archive, Paper 2016/116, 2016; published version: Springer, 2016, pp. 31–60. eprint.iacr.org/2016/116.

[15] A. R. Block, A. Garreta, J. Katz, J. Thaler, P. R. Tiwari and M. Zając, "Fiat-Shamir Security of FRI and Related SNARKs," ASIACRYPT 2023; Cryptology ePrint Archive, Paper 2023/1071. eprint.iacr.org/2023/1071.

[16] E. Ben-Sasson, "ethSTARK Documentation," Cryptology ePrint Archive, Paper 2021/582, 2021. eprint.iacr.org/2021/582.

[17] I. S. Reed and G. Solomon, "Polynomial Codes over Certain Finite Fields," Journal of the Society for Industrial and Applied Mathematics, vol. 8, no. 2, 1960, pp. 300–304. doi.org/10.1137/0108018.

[18] R. C. Merkle, “A Digital Signature Based on a Conventional Encryption Function,” in Advances in Cryptology — CRYPTO ’87, Lecture Notes in Computer Science, vol. 293, Springer, 1988, pp. 369–378. doi.org/10.1007/3-540-48184-2_32.

[19] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty and S. Saraf, “Proximity Gaps for Reed-Solomon Codes,” FOCS 2020; Cryptology ePrint Archive, Paper 2020/654. eprint.iacr.org/2020/654.

[20] M. Al-Bassam, A. Sonnino and V. Buterin, “Fraud and Data Availability Proofs: Maximising Light Client Security and Scaling Blockchains with Dishonest Majorities,” arXiv:1809.09044, 2018. arxiv.org/abs/1809.09044.

[21] M. Hall-Andersen, M. Simkin and B. Wagner, “Foundations of Data Availability Sampling,” IACR Communications in Cryptology, 2024; Cryptology ePrint Archive, Paper 2023/1079. doi.org/10.62056/a09qudhdj.

[22] C. Dwork and M. Naor, “Pricing via Processing or Combatting Junk Mail,” in Advances in Cryptology — CRYPTO ’92, Lecture Notes in Computer Science, vol. 740, Springer, 1992, pp. 139–147.

[23] M. Ball, A. Rosen, M. Sabin and P. N. Vasudevan, “Proofs of Useful Work,” Cryptology ePrint Archive, Paper 2017/203, 2017; updated version: “Proofs of Work from Worst-Case Assumptions,” CRYPTO 2018, pp. 789–819. eprint.iacr.org/2017/203.

[24] M. Fitzi, A. Kiayias, G. Panagiotakos and A. Russell, “Ofelimos: Combinatorial Optimization via Proof-of-Useful-Work. A Provably Secure Blockchain Protocol,” Cryptology ePrint Archive, Paper 2021/1379, 2021. eprint.iacr.org/2021/1379.

[25] I. Komargodski and O. Weinstein, “Proofs of Useful Work from Arbitrary Matrix Multiplication,” arXiv:2504.09971, version v4 of 13 November 2025; version v1 also with I. Schen. arxiv.org/abs/2504.09971.

[26] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. bitcoin.org/bitcoin.pdf.

[27] J. A. Garay, A. Kiayias and N. Leonardos, “The Bitcoin Backbone Protocol: Analysis and Applications,” EUROCRYPT 2015; Journal of the ACM, vol. 71, no. 4, 2024, pp. 25:1–25:49; Cryptology ePrint Archive, Paper 2014/765. eprint.iacr.org/2014/765.

[28] J. A. Garay, A. Kiayias and N. Leonardos, “The Bitcoin Backbone Protocol with Chains of Variable Difficulty,” CRYPTO 2017; Cryptology ePrint Archive, Paper 2016/1048. eprint.iacr.org/2016/1048.

[29] C. Decker and R. Wattenhofer, “Information Propagation in the Bitcoin Network,” in 13th IEEE International Conference on Peer-to-Peer Computing (P2P), Trento, 2013.

[30] R. Pass, L. Seeman and a. shelat, “Analysis of the Blockchain Protocol in Asynchronous Networks,” Cryptology ePrint Archive, Paper 2016/454, 2016. eprint.iacr.org/2016/454.

[31] A. Dembo, S. Kannan, E. N. Tas, D. Tse, P. Viswanath, X. Wang and O. Zeitouni, “Everything is a Race and Nakamoto Always Wins,” arXiv:2005.10484, 2020. arxiv.org/abs/2005.10484.

[32] P. Gaži, A. Kiayias and A. Russell, “Tight Consistency Bounds for Bitcoin,” ACM CCS 2020; Cryptology ePrint Archive, Paper 2020/661. eprint.iacr.org/2020/661.

[33] I. Eyal and E. G. Sirer, “Majority is not Enough: Bitcoin Mining is Vulnerable,” arXiv:1311.0243, 2013. arxiv.org/abs/1311.0243.

[34] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach and A. Juels, “Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges,” arXiv:1904.05234, 2019. arxiv.org/abs/1904.05234.

[35] T. Roughgarden, “Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559,” arXiv:2012.00854, 2020. arxiv.org/abs/2012.00854.

[36] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer and M. Virza, “Zerocash: Decentralized Anonymous Payments from Bitcoin,” 2014 IEEE Symposium on Security and Privacy; Cryptology ePrint Archive, Paper 2014/349. eprint.iacr.org/2014/349.

[37] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry and A. Gervais, “SoK: Layer-Two Blockchain Protocols,” Financial Cryptography and Data Security 2020; Cryptology ePrint Archive, Paper 2019/360. eprint.iacr.org/2019/360.

[38] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, August 2024. csrc.nist.gov/pubs/fips/203/final.

[39] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard," FIPS 204, August 2024. csrc.nist.gov/pubs/fips/204/final.

[40] National Institute of Standards and Technology, "Stateless Hash-Based Digital Signature Standard," FIPS 205, August 2024. csrc.nist.gov/pubs/fips/205/final.

[41] National Institute of Standards and Technology, "Recommendations for Key-Encapsulation Mechanisms," SP 800-227, September 2025. csrc.nist.gov/pubs/sp/800/227/final.

[42] National Institute of Standards and Technology, "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," FIPS 202, August 2015. csrc.nist.gov/pubs/fips/202/final.

[43] National Institute of Standards and Technology, "SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash and ParallelHash," SP 800-185, December 2016. csrc.nist.gov/pubs/sp/800/185/final.

[44] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, May 2010. rfc-editor.org/rfc/rfc5869.

[45] National Institute of Standards and Technology, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," SP 800-38D, November 2007. csrc.nist.gov/pubs/sp/800/38/d/final.

[46] National Institute of Standards and Technology, "Post-Quantum Cryptography" project. csrc.nist.gov/projects/post-quantum-cryptography.

[47] L. K. Grover, "A fast quantum mechanical algorithm for database search," in Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC 1996), 1996, pp. 212–219. arxiv.org/abs/quant-ph/9605043.

[48] G. Brassard, P. Høyer and A. Tapp, "Quantum Algorithm for the Collision Problem," arXiv:quant-ph/9705002, 1997. arxiv.org/abs/quant-ph/9705002.

[49] P. W. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," SIAM Journal on Computing, vol. 26, no. 5, 1997, pp. 1484–1509.

[50] J. Don, S. Fehr and C. Majenz, "The Measure-and-Reprogram Technique 2.0: Multi-Round Fiat-Shamir and More," arXiv:2003.05207, 2020. arxiv.org/abs/2003.05207.

[51] M. O. Rabin, "Probabilistic Algorithms in Finite Fields," SIAM Journal on Computing, vol. 9, no. 2, 1980, pp. 273–280.

[52] A. R. Block and P. R. Tiwari, "On the Concrete Security of Non-interactive FRI," Cryptology ePrint Archive, Paper 2024/1161, 2024. eprint.iacr.org/2024/1161.

[53] M. L. Balinski and H. P. Young, "Fair Representation: Meeting the Ideal of One Man, One Vote," Yale University Press, 1982; 2nd ed., Brookings Institution Press, 2001.